■ Privacy-First AI Clinical Decision Support Platform

End-to-End LLM • RAG • AI Agent • Amazon Bedrock Learning Assignment

Project vision

Build a real-world-style clinical AI platform using **synthetic patient data** to learn LLMs, RAG, AI agents, Amazon Bedrock, privacy, security, evaluation, and production AI architecture.

Safety boundary: This is a learning/clinical-decision-support prototype. It must not provide autonomous medical diagnosis, replace a qualified clinician, or be tested with real patient data during the learning phase.

Guiding principle: Use Amazon Bedrock for the core AI learning and spend AWS credits primarily on LLM/agent/RAG capabilities. Use free or low-cost cloud services for commodity supporting infrastructure wherever practical.

Learning rule: Build one layer → deploy → test → understand → document → then add the next layer.

PART 1 — End-to-End Bedrock Connectivity

■ **Goal:** Get UI → .NET → Bedrock → model → response working.

■■ Technology Stack

• React / Next.js

• ASP.NET Core / .NET 8

• Amazon Bedrock

• AWS SDK for .NET

• Terraform

• Git + GitHub

• xUnit

Architecture

```
React / Next.js → ASP.NET Core → Amazon Bedrock → Foundation Model → Response
```

PART 2 — Patient Application

■ **Goal:** Introduce a synthetic patient domain and generate clinical summaries and considerations without autonomous diagnosis.

■■ Technology Stack

• React / Next.js

• ASP.NET Core

• C# / Clean Architecture

• Supabase PostgreSQL (Free)

• Entity Framework Core

• REST / OpenAPI

• Amazon Bedrock

Architecture

```
Doctor → Patient → .NET API → Bedrock → Clinical Summary
```

PART 3 — Authentication, Privacy & Security

■ **Goal:** Protect patient information before adding advanced AI capabilities.

■■ Technology Stack

• Supabase Auth

• JWT + RBAC

• AWS IAM

• AWS KMS

• Platform secrets initially

• HTTPS/TLS

• PII/PHI redaction

• PostgreSQL audit events

• Terraform

Architecture

```
User → Authentication → Authorization → Privacy Layer → AI
```

PART 4 — Medical Document Management

■ **Goal:** Process synthetic reports, clinical guidelines and research documents.

■■ Technology Stack

• Cloudflare R2

• PDF / DOCX / TXT

• PyMuPDF / python-docx

• Python document processor

• ASP.NET Core

• PostgreSQL metadata

• Later: Amazon S3

Architecture

```
Document → R2 → Document Processor → Extracted Text → RAG Pipeline
```

PART 5 — Custom RAG

■ **Goal:** Understand RAG internals before using managed AWS RAG.

■■ Technology Stack

• Python

• PyMuPDF / python-docx

• Python chunking

• Hugging Face / Sentence Transformers

• Qdrant Cloud Free

• Amazon Bedrock

• PostgreSQL metadata

• Python evaluation

Architecture

```
Documents → Chunking → Embeddings → Qdrant → Similarity Search → Top-K Context → Bedrock → Answer
```

PART 6 — AWS-Native RAG

■ **Goal:** Replace the custom pipeline with Bedrock Knowledge Bases and compare both approaches.

■■ Technology Stack

• Amazon Bedrock Knowledge Bases

• Amazon S3

• Bedrock Embeddings

• Managed retrieval/vector layer

• Amazon Bedrock

• Terraform

Architecture

```
Documents → S3 → Bedrock Knowledge Bases → Embeddings/Retrieval → Bedrock → Answer
```

PART 7 — Evidence-Based Clinical Response

■ **Goal:** Separate patient facts, retrieved evidence, AI interpretation and uncertainty.

■■ Technology Stack

• Bedrock

• Bedrock Knowledge Bases

• Structured prompts

• JSON Schema / structured output

• Retrieved-document metadata

• .NET + Python validation

• Bedrock/custom evaluation

Architecture

```
Patient Facts + Retrieved Evidence + AI Interpretation + Uncertainty → Clinical Response
```

PART 8 — AI Agent

■ **Goal:** Move from chatbot to an agent capable of selecting and invoking tools.

■■ Technology Stack

• Amazon Bedrock Agents

• Bedrock

• Agent Actions / tool calling

• ASP.NET Core APIs

• .NET tools

• Bedrock Knowledge Bases

• Supabase

• R2 / S3

• Terraform

Architecture

```
Doctor → Agent → Patient / Lab / RAG / Report Tools → Bedrock → Response
```

PART 9 — Agent Memory

■ **Goal:** Introduce short-term and long-term memory with retention and access controls.

■■ Technology Stack

• Conversation context

• Supabase PostgreSQL

• Qdrant / Knowledge Base

• Upstash Redis

• Bedrock Agents

• AWS KMS

• Retention policies

Architecture

```
Conversation + Patient Context + Clinical History + Working Memory
```

PART 10 — AI Safety & Guardrails

■ **Goal:** Prevent unsafe, unsupported or unauthorized AI behavior.

■■ Technology Stack

• Amazon Bedrock Guardrails

• Application prompt validation

• .NET output validation

• Application policy rules

• PII filtering/redaction

• RBAC

• PostgreSQL / CloudWatch audit

• Grafana

Architecture

```
Bedrock → AI Response → Guardrails + Rules + Validation → Doctor Response
```

PART 11 — LLM/RAG Evaluation

■ **Goal:** Measure retrieval quality, answer quality, hallucinations, latency, tokens and cost.

■■ Technology Stack

• Amazon Bedrock evaluation

• Python evaluation

• JSON/CSV datasets

• Retrieval + generation metrics

• MLflow / simple DB

• pytest + xUnit

Architecture

```
Evaluation Dataset → Model/RAG Run → Metrics → Compare Models, Prompts and Retrievers
```

PART 12 — Production AWS Architecture

■ **Goal:** Move the learning system toward production-style AWS architecture.

■■ Technology Stack

• Next.js + CloudFront/Vercel

• AWS WAF, IAM, KMS, Cognito

• API Gateway + ASP.NET Core/Lambda

• Bedrock + Agents + Knowledge Bases + Guardrails

• S3 + DynamoDB/Aurora

• Knowledge Bases / OpenSearch

• EventBridge / SQS

• CloudWatch + OpenTelemetry + Grafana

• Terraform

• GitHub Actions

Architecture

```
Users → WAF/CloudFront → API → Auth → .NET → Privacy/Audit → Agent → Bedrock/RAG/Tools → Safety →
Clinician
```

PART 13 — Advanced AI

■ **Goal:** Explore advanced agentic architectures only after the fundamentals are stable.

■■ Technology Stack

• Multi-agent architecture

• MCP

• Advanced tool calling

• Agent memory

• Agent evaluation

• Model routing/fallback

• Prompt optimization

• Cost optimization

- AI observability
- Human-in-the-loop workflows

Architecture

```
Clinical Agent ↔ Research Agent ↔ Tools/RAG → Bedrock → Safety → Human Clinician
```

■ AWS Free-Credit Strategy

Primary rule: AWS credits should be concentrated on the AI capabilities that are the core learning objective.

Priority	Use	Strategy
■ High	Bedrock / Agents / Knowledge Bases / Guardrails / Embeddings / Evaluation	Spend credits here; these are the main learning targets.
■ Free/Low	Supabase / R2 / Qdrant / Vercel / Upstash / Grafana Cloud	Use free tiers for commodity infrastructure.
■ Later	S3 / DynamoDB / OpenSearch / Cognito / Lambda / API Gateway	Introduce when the AWS service itself becomes the next lesson.
■ Avoid initially	EC2 / NAT Gateway / EKS / always-on infrastructure	Can consume credits without adding much to initial AI learning.

■ AWS Replacement Learning

Initial low-cost service	AWS equivalent / later lesson
Supabase PostgreSQL	Aurora PostgreSQL / DynamoDB
Cloudflare R2	Amazon S3
Qdrant Cloud	Bedrock Knowledge Bases / OpenSearch
Supabase Auth	Amazon Cognito
Upstash Redis	AWS managed caching
Grafana Cloud	CloudWatch / OpenTelemetry
Vercel / Cloudflare	CloudFront + AWS hosting

■■ Final Learning Journey

1. End-to-End Bedrock → 2. .NET + Bedrock API → 3. Synthetic Patient Data → 4. Privacy + Security → 5. Document Processing → 6. Custom RAG + Qdrant → 7. Bedrock Knowledge Bases → 8. Evidence-Based AI → 9. Custom Agent → 10. Bedrock Agents → 11. Memory + Tools → 12. Guardrails + Safety → 13. Evaluation → 14. Production AWS → 15. Multi-Agent / Advanced AI

■ Definition of Success

By the end, the learner should be able to explain and implement the full path from a basic Bedrock model invocation to a privacy-aware, evaluated, tool-using clinical AI agent, and understand when to use a managed AWS service versus a lower-cost supporting alternative.

■■ Safety & Privacy Requirements

Use synthetic data throughout the learning project. Do not upload real patient records. Enforce authentication, authorization, data minimization, encryption, controlled retention, auditability and safe handling of AI uncertainty. The AI remains a clinical decision-support component and not an autonomous diagnostic authority.